

UNIVERSIDADE TUIUTI DO PARANÁ

**JOÃO THOMAZ VIEIRA
KLAUS SIEGFRIED BECKMANN**

**FERRAMENTA BASEADA EM LINGUAGEM DE DOMÍNIO
ESPECÍFICO PARA O APOIO AO ENSINO DE AUTÔMATOS FINITOS
COM SIMULAÇÃO VISUAL INTERATIVA**

**CURITIBA
2026**

**JOÃO THOMAZ VIEIRA
KLAUS SIEGFRIED BECKMANN**

**FERRAMENTA BASEADA EM LINGUAGEM DE DOMÍNIO
ESPECÍFICO PARA O APOIO AO ENSINO DE AUTÔMATOS FINITOS
COM SIMULAÇÃO VISUAL INTERATIVA**

Trabalho de Conclusão de Curso apresentado ao curso de Bacharelado em Ciência da Computação da Faculdade de Ciências Exatas e de Tecnologia da Universidade Tuiuti do Paraná, como requisito à obtenção ao grau de Bacharel.

Orientador: Prof. Diógenes Cogo Furlan

**CURITIBA
2026**

AGRADECIMENTOS

Agradecemos a Manoel Gomes, o eterno Caneta Azul, cujo meme nos acompanhou nos momentos mais difíceis desta jornada acadêmica. Sua obra, de profundidade imensurável, foi a inspiração diária que nos deu forças para continuar.

RESUMO

O ensino de autômatos finitos em cursos de Ciência da Computação enfrenta dificuldades relacionadas à abstração dos conceitos teóricos, uma vez que as ferramentas existentes adotam abordagens exclusivamente visuais, sem oferecer representação textual formal, versionamento ou diagnósticos de erro precisos. Este trabalho propõe o desenvolvimento de uma ferramenta educacional composta por uma Linguagem de Domínio Específico (DSL) textual para a descrição declarativa de Autômatos Finitos Determinísticos (AFD) e Não-Determinísticos (AFN), acompanhada de um compilador que realiza análise léxica, sintática e semântica, fornecendo mensagens de erro com indicação de localização e natureza do problema. A ferramenta integra ainda uma interface gráfica interativa que renderiza visualmente o autômato descrito e anima o processo de simulação passo a passo, destacando os estados e transições percorridos durante o reconhecimento de cadeias de entrada. A validação do protótipo é realizada por meio de testes com autômatos clássicos da literatura, incluindo casos com erros propositais para avaliar a qualidade dos diagnósticos gerados. Espera-se que a ferramenta contribua para a compreensão prática do funcionamento de autômatos finitos, conectando a teoria de autômatos à prática de compiladores em um único instrumento educacional.

Palavras-chave: Autômatos Finitos; Linguagem de Domínio Específico; Compiladores; Simulação Visual; Ensino de Computação.

LISTA DE FIGURAS

FIGURA 1 – Árvore de derivação da palavra <i>aabb</i> na gramática G	14
FIGURA 2 – AFD que reconhece cadeias contendo 01 como subcadeia	16
FIGURA 3 – AFN que reconhece cadeias terminadas em 01	18
FIGURA 4 – AFD construído a partir do AFN da Figura 3 pela construção de subconjuntos	20

LISTA DE TABELAS

TABELA 1 – Hierarquia de Chomsky	14
TABELA 2 – Tabela de transição do AFD que reconhece cadeias contendo 01 .	21

LISTA DE SIGLAS E ACRÔNIMOS

AFD	Autômato Finito Determinístico
AFN	Autômato Finito Não-Determinístico
AST	Árvore Sintática Abstrata
BNF	Forma de Backus-Naur
DSL	Linguagem de Domínio Específico
EBNF	Forma de Backus-Naur Estendida

LISTA DE SÍMBOLOS

Σ	Alfabeto
Σ^*	Conjunto de todas as palavras sobre Σ
ε	Palavra vazia
$ w $	Comprimento da palavra w
$\emptyset$	Conjunto vazio
$\in$	Pertence a
$\notin$	Não pertence a
$\subseteq$	Subconjunto
$\cup$	União
$\cap$	Interseção
$\mathcal{P}(Q)$	Conjunto das partes de Q
L^*	Fecho de Kleene da linguagem L
$\rightarrow$	Regra de produção
$\Rightarrow$	Derivação direta
$\Rightarrow^*$	Derivação em zero ou mais passos
δ	Função de transição
$\hat{\delta}$	Função de transição estendida a cadeias
$L(M)$	Linguagem reconhecida pelo autômato M
$L(G)$	Linguagem gerada pela gramática G

SUMÁRIO

1	INTRODUÇÃO	9
2	AUTÔMATOS FINITOS E LINGUAGENS FORMAIS	11
2.1	CONCEITOS FUNDAMENTAIS DE LINGUAGENS FORMAIS	11
2.2	GRAMÁTICAS FORMAIS	12
2.3	AUTÔMATO FINITO DETERMINÍSTICO	15
2.4	AUTÔMATO FINITO NÃO-DETERMINÍSTICO	16
2.5	EQUIVALÊNCIA ENTRE AFD E AFN	18
2.6	REPRESENTAÇÕES DE AUTÔMATOS	20
2.7	SIMULAÇÃO DE AUTÔMATOS	22
2.8	APLICAÇÕES DE AUTÔMATOS	24
3	COMPILADORES E LINGUAGENS DE DOMÍNIO ESPECÍFICO	26
3.1	CONCEITOS DE COMPILADORES	26
3.2	ANÁLISE LÉXICA	26
3.3	ANÁLISE SINTÁTICA	26
3.4	ANÁLISE SEMÂNTICA	26
3.5	AST (ÁRVORE SINTÁTICA ABSTRATA)	26
3.6	LINGUAGENS DE DOMÍNIO ESPECÍFICO (DSL)	26
3.7	CONSTRUÇÃO DE COMPILADORES (FERRAMENTAS)	26
3.8	DIAGNÓSTICO DE ERROS	26
3.9	DSL NO ENSINO	26
4	RUST	27
	REFERÊNCIAS	28

1 INTRODUÇÃO

O ensino de Teoria da Computação, em especial o conteúdo de autômatos finitos, representa um desafio recorrente nos cursos de Bacharelado em Ciência da Computação. Conceitos como estados, transições, alfabetos e reconhecimento de cadeias são abstratos por natureza, e sua compreensão prática depende fortemente da capacidade do estudante de visualizar e simular o comportamento das máquinas formais descritas teoricamente em sala de aula.

As ferramentas disponíveis atualmente para apoiar esse aprendizado, como o JFLAP (RODGER; FINLEY, 2006), adotam uma abordagem exclusivamente visual baseada em arrastar e soltar, na qual o usuário constrói o autômato posicionando estados e conectando transições com o *mouse*. Embora essa abordagem seja útil para ilustrar conceitos, ela apresenta limitações relevantes: a descrição do autômato não é textual, o que dificulta o versionamento e o compartilhamento dos modelos; não há validação formal da estrutura com mensagens de erro claras e localizadas; e o estudante que comete um erro na construção não recebe um diagnóstico preciso do problema.

Diante desse cenário, este trabalho propõe o desenvolvimento de uma ferramenta educacional que adota uma abordagem diferente. A solução consiste em uma Linguagem de Domínio Específico (DSL) textual para a descrição declarativa de Autômatos Finitos Determinísticos (AFD) e Não-Determinísticos (AFN), acompanhada de um compilador que realiza análise léxica, sintática e semântica, fornecendo mensagens de erro com indicação precisa da localização e da natureza do problema. A ferramenta integra ainda uma interface gráfica interativa que renderiza visualmente o autômato descrito e anima o processo de simulação passo a passo, destacando os estados e transições percorridos durante o reconhecimento de cadeias de entrada.

Um aspecto adicional de relevância é que a própria ferramenta, por ser construída com técnicas reais de compilação, funciona como um exemplo prático dos conceitos que o estudante está aprendendo, conectando a teoria de autômatos à prática de compiladores em um único instrumento educacional. A ausência de uma ferramenta que integre descrição textual formal, validação com diagnósticos claros e simulação visual interativa justifica o desenvolvimento deste trabalho, que busca preencher essa lacuna com uma solução unificada e pedagogicamente orientada.

O objetivo geral deste trabalho é desenvolver uma ferramenta educacional baseada em uma Linguagem de Domínio Específico para apoiar o ensino de autômatos finitos, com simulação visual interativa. Para atingir esse objetivo, definem-se os seguintes objetivos específicos:

- a) projetar uma DSL textual com sintaxe própria para a descrição declarativa de AFDs e AFNs, com a gramática formalizada em notação BNF/EBNF;

- b) implementar um compilador para a DSL com análise léxica, sintática e semântica, construindo uma Árvore Sintática Abstrata (AST) e fornecendo mensagens de erro com indicação de localização e natureza do problema;
- c) desenvolver um simulador capaz de processar cadeias de entrada sobre os autômatos descritos, determinando aceitação ou rejeição e registrando o caminho percorrido;
- d) construir uma interface gráfica interativa que renderize o autômato e anime a simulação passo a passo, permitindo ao usuário controlar a velocidade e navegar entre os passos;
- e) validar o protótipo por meio de testes com autômatos clássicos da literatura, incluindo casos com erros propositalmente para avaliar a qualidade dos diagnósticos gerados.

Este trabalho está organizado como segue. O Capítulo 2 apresenta os conceitos teóricos de autômatos finitos e linguagens formais. O Capítulo 3 aborda linguagens de domínio específico e as fases de compilação. O Capítulo ?? discute as ferramentas relacionadas e suas limitações. O Capítulo ?? descreve o projeto da DSL, sua sintaxe e gramática. O Capítulo ?? detalha a implementação do compilador e da interface gráfica. O Capítulo ?? apresenta os testes e os resultados obtidos. Por fim, o Capítulo ?? apresenta as considerações finais e trabalhos futuros.

2 AUTÔMATOS FINITOS E LINGUAGENS FORMAIS

Este capítulo apresenta os fundamentos teóricos necessários à compreensão dos autômatos finitos e das linguagens regulares, formalismos que constituem o objeto central da ferramenta proposta neste trabalho. Inicia-se com os conceitos básicos de linguagens formais (Seção 2.1) e gramáticas (Seção 2.2), seguindo-se a formalização dos autômatos finitos determinísticos (Seção 2.3) e não-determinísticos (Seção 2.4). Por fim, demonstra-se a equivalência entre ambos os modelos (Seção 2.5), resultado que justifica o uso de qualquer um deles na descrição de linguagens regulares.

2.1 CONCEITOS FUNDAMENTAIS DE LINGUAGENS FORMAIS

O estudo dos autômatos finitos exige a compreensão prévia de algumas estruturas matemáticas elementares: conjuntos, alfabetos, palavras e linguagens. Esta seção apresenta tais conceitos de forma sistemática, fixando a notação e a terminologia adotadas nos capítulos subsequentes.

Um **conjunto** é uma coleção de elementos, que podem ser de qualquer natureza, como números, letras, símbolos ou mesmo outros conjuntos. Para indicar que um elemento a pertence a um conjunto C , escreve-se $a \in C$; o símbolo $\notin$ denota a relação inversa. Por exemplo, dado $C = \{a, b, c\}$, tem-se $a \in C$ e $d \notin C$. Conjuntos podem ser finitos ou infinitos; no segundo caso, é usual o emprego de reticências para indicar a continuação dos elementos, como em $\mathbb{Z} = \{\dots, -2, -1, 0, 1, 2, \dots\}$. O conjunto que não contém qualquer elemento é denominado **conjunto vazio** e é representado pelo símbolo $\emptyset$ (SIPSER, 2012).

A partir desta noção de conjunto, define-se a primeira estrutura específica da teoria das linguagens formais: o alfabeto. Um **alfabeto** é um conjunto finito e não vazio de símbolos, representado, por convenção, pela letra grega Σ (SIPSER, 2012). Os elementos de um alfabeto são denominados símbolos ou letras. Por exemplo, $\Sigma_1 = \{0, 1\}$ é o alfabeto binário, enquanto $\Sigma_2 = \{a, b, c, \dots, z\}$ corresponde ao alfabeto latino minúsculo. A exigência de finitude é fundamental, pois garante que os autômatos, definidos nas seções seguintes, possam manipular o alfabeto de forma efetiva; a não vacuidade, por sua vez, assegura que sempre haja ao menos um símbolo disponível para a formação de palavras (VIEIRA, 2006; DIVERIO; MENEZES, 2011).

Sobre um alfabeto Σ , define-se uma **palavra** (ou cadeia) como uma sequência finita de símbolos pertencentes a Σ , escritos de forma justaposta e sem separadores. O comprimento de uma palavra w , denotado por $|w|$, corresponde ao número de símbolos que a compõem. A palavra de comprimento zero, denominada **palavra vazia**, é representada pelo símbolo ε (SIPSER, 2012). O conjunto de todas as palavras que podem ser formadas sobre Σ , incluindo a palavra vazia, é denotado por Σ^* . Assim, para

$\Sigma = \{a, b\}$, tem-se $\Sigma^* = \{\varepsilon, a, b, aa, ab, ba, bb, aaa, \dots\}$, conjunto que é sempre infinito e enumerável (VIEIRA, 2006).

Estabelecidas as noções de alfabeto e palavra, é possível definir o objeto central deste trabalho: a linguagem. Uma **linguagem formal** sobre um alfabeto Σ é qualquer subconjunto de Σ^* , ou seja, qualquer conjunto de palavras formadas a partir dos símbolos de Σ (SIPSER, 2012; VIEIRA, 2006). Linguagens podem ser finitas, como $L_1 = \{a, ab, abb\}$, ou infinitas, como $L_2 = \{a^n b^n \mid n \geq 1\}$, que contém todas as palavras compostas por uma sequência de a 's seguida de igual número de b 's. A maior parte das linguagens de interesse teórico e prático é infinita, o que motiva o desenvolvimento de mecanismos formais, como gramáticas e autômatos, capazes de descrevê-las de maneira finita.

Por serem conjuntos, as linguagens admitem as operações usuais de **união**, **interseção** e **diferença**. Além dessas, três operações específicas, derivadas da estrutura sequencial das palavras, desempenham papel central na teoria (VIEIRA, 2006; DIVERIO; MENEZES, 2011). A **concatenação** de duas palavras $x = a_1 a_2 \dots a_m$ e $y = b_1 b_2 \dots b_n$ produz a palavra $xy = a_1 a_2 \dots a_m b_1 b_2 \dots b_n$; em particular, $w\varepsilon = \varepsilon w = w$ para qualquer palavra w , o que faz de ε o elemento neutro dessa operação. A concatenação estende-se naturalmente a linguagens: dadas L_1 e L_2 , define-se $L_1 L_2 = \{xy \mid x \in L_1 \text{ e } y \in L_2\}$. Por fim, o **fecho de Kleene** de uma linguagem L , denotado por L^* , corresponde ao conjunto de todas as palavras obtidas pela concatenação de zero ou mais elementos de L , isto é, $L^* = \bigcup_{n \in \mathbb{N}} L^n$, onde $L^0 = \{\varepsilon\}$ e $L^n = L^{n-1} L$ para $n \geq 1$. Essas operações constituem a base para a especificação concisa de linguagens, recurso amplamente explorado nas expressões regulares e gramáticas estudadas nas seções seguintes.

2.2 GRAMÁTICAS FORMAIS

Conforme observado na seção anterior, a maior parte das linguagens de interesse prático é infinita, o que torna inviável descrevê-las pela enumeração explícita de suas palavras. Faz-se necessário, portanto, um mecanismo finito capaz de gerar todas as palavras de uma linguagem. Entre os formalismos propostos para esse fim, as **gramáticas** ocupam posição de destaque, tanto pelo seu poder expressivo quanto pela estreita relação com os autômatos estudados nas seções subsequentes (VIEIRA, 2006; LEWIS; PAPADIMITRIOU, 1998).

Uma gramática descreve uma linguagem por meio de um conjunto finito de **regras de produção**, que indicam como certas sequências de símbolos podem ser substituídas por outras. Nessa estrutura, distinguem-se dois tipos de símbolos: os **terminais**, que compõem o alfabeto da linguagem gerada e aparecem nas palavras finais, e os **não-terminais** (ou **variáveis**), símbolos auxiliares que servem de interme-

diários no processo de geração e não fazem parte das palavras produzidas (LEWIS; PAPADIMITRIOU, 1998; SIPSER, 2012). A geração de uma palavra inicia-se sempre a partir de uma variável especial, denominada **símbolo inicial** (ou variável de partida), e prossegue pela aplicação sucessiva das regras até que reste apenas uma sequência de terminais.

Formalmente, uma **gramática** é uma quádrupla $G = (V, \Sigma, R, S)$, na qual (VIEIRA, 2006; LEWIS; PAPADIMITRIOU, 1998):

- V é um conjunto finito de variáveis (símbolos não-terminais);
- Σ é um alfabeto de símbolos terminais, com $V \cap \Sigma = \emptyset$;
- R é um conjunto finito de **regras de produção**, cada uma escrita na forma $\alpha \rightarrow \beta$, em que $\alpha, \beta \in (V \cup \Sigma)^*$ e o lado esquerdo α contém ao menos uma variável;
- $S \in V$ é a variável de partida.

A aplicação de uma regra $\alpha \rightarrow \beta \in R$ a uma cadeia de variáveis e terminais produz uma nova cadeia pela substituição de uma ocorrência de α por β . Essa relação de substituição direta é denotada por $\Rightarrow$: dadas $u, v \in (V \cup \Sigma)^*$, escreve-se $u \Rightarrow v$ quando existem $x, y \in (V \cup \Sigma)^*$ e uma regra $\alpha \rightarrow \beta \in R$ tais que $u = x\alpha y$ e $v = x\beta y$ (LEWIS; PAPADIMITRIOU, 1998). Em outras palavras, x e y representam o trecho da cadeia que permanece inalterado, enquanto a subcadeia α , situada entre eles, é trocada por β . A título de exemplo, dada a regra $A \rightarrow ab$ e a cadeia cAd , tem-se $cAd \Rightarrow cabd$, identificando-se $x = c$, $\alpha = A$, $y = d$ e $\beta = ab$.

A aplicação sucessiva de regras, isto é, o fecho reflexivo e transitivo de $\Rightarrow$, é denotada por $\Rightarrow^*$. A **linguagem gerada** pela gramática G é o conjunto de todas as palavras formadas exclusivamente por terminais que podem ser obtidas a partir do símbolo inicial: $L(G) = \{w \in \Sigma^* \mid S \Rightarrow^* w\}$ (VIEIRA, 2006; SIPSER, 2012).

A título de ilustração, considere a gramática $G = (\{S\}, \{a, b\}, R, S)$, em que $R = \{S \rightarrow aSb, S \rightarrow \varepsilon\}$. Partindo do símbolo inicial S , é possível obter, por exemplo, a palavra $aabb$ por meio da seguinte derivação:

$$\begin{array}{ll}
 S \Rightarrow aSb & \text{(aplicação da regra } S \rightarrow aSb\text{)} \\
 \Rightarrow aaSbb & \text{(aplicação da regra } S \rightarrow aSb\text{)} \\
 \Rightarrow aabb & \text{(aplicação da regra } S \rightarrow \varepsilon\text{)}
 \end{array}$$

A estrutura dessa derivação pode ser visualizada por meio de uma **árvore de derivação**, representada na Figura 1, na qual a raiz corresponde ao símbolo inicial, os nós internos representam as variáveis substituídas e as folhas correspondem aos terminais que compõem a palavra gerada (SIPSER, 2012).

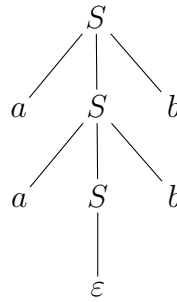


FIGURA 1 – Árvore de derivação da palavra *aabb* na gramática *G*

FONTE: Os autores (2026)

De modo geral, cada aplicação da regra $S \rightarrow aSb$ acrescenta um a à esquerda e um b à direita do símbolo S , enquanto a regra $S \rightarrow \varepsilon$ encerra a derivação eliminando a variável remanescente. Conclui-se que $L(G) = \{a^n b^n \mid n \geq 0\}$, linguagem que, conforme será discutido adiante, não pode ser gerada por nenhum mecanismo mais restrito que uma gramática livre de contexto (LEWIS; PAPADIMITRIOU, 1998; SIPSER, 2012).

Restrições impostas sobre a forma das regras de produção dão origem a diferentes classes de gramáticas e, conseqüentemente, a diferentes classes de linguagens. Essa estratificação, conhecida como **Hierarquia de Chomsky**, organiza as linguagens em quatro categorias (irrestritas, sensíveis ao contexto, livres de contexto e regulares), cada uma associada a um modelo computacional específico (VIEIRA, 2006; LEWIS; PAPADIMITRIOU, 1998). O Quadro 1 sintetiza essa classificação, evidenciando a forma das regras de produção admitidas em cada classe e o modelo computacional que reconhece a linguagem correspondente.

TABELA 1 – Hierarquia de Chomsky

Tipo	Classe de Linguagem	Forma das Regras	Modelo Computacional
0	Recursivamente enumeráveis	$\alpha \rightarrow \beta$	Máquina de Turing
1	Sensíveis ao contexto	$\alpha A \beta \rightarrow \alpha \gamma \beta$	Autômato linearmente limitado
2	Livres de contexto	$A \rightarrow \gamma$	Autômato com pilha
3	Regulares	$A \rightarrow aB$ ou $A \rightarrow a$	Autômato finito

FONTE: Adaptado de Vieira (2006) e Lewis e Papadimitriou (1998)

As gramáticas regulares (Tipo 3), em particular, geram exatamente a classe das linguagens reconhecidas pelos autômatos finitos, formalismo apresentado nas próximas seções.

2.3 AUTÔMATO FINITO DETERMINÍSTICO

Conforme indicado ao final da seção anterior, as gramáticas regulares (Tipo 3 da Hierarquia de Chomsky) descrevem exatamente a classe das linguagens reconhecidas pelo modelo computacional mais simples: o autômato finito. Esta seção apresenta a formalização do **autômato finito determinístico** (AFD), modelo que serve como ponto de partida para o estudo dos reconhecedores de linguagens regulares e que constitui o objeto central da ferramenta proposta neste trabalho.

Intuitivamente, um AFD é uma máquina abstrata composta por um número finito de **estados**, interligados por **transições** rotuladas com símbolos de um alfabeto. A máquina inicia sua operação em um estado especial, denominado **estado inicial**, e processa uma cadeia de entrada lendo seus símbolos um a um, da esquerda para a direita. A cada símbolo lido, o autômato transita para um novo estado, conforme determinado pela rotulação das arestas. Encerrado o processamento da cadeia, se o autômato encontrar-se em um dos estados designados como **estados finais** (ou de aceitação), diz-se que a cadeia foi **aceita**; caso contrário, é **rejeitada** (HOPCROFT *et al.*, 2006; SIPSER, 2012). O termo *determinístico* reflete a característica de que, para cada combinação de estado atual e símbolo de entrada, existe uma única transição possível.

Formalmente, um **autômato finito determinístico** é uma quintupla $M = (Q, \Sigma, \delta, q_0, F)$, na qual (HOPCROFT *et al.*, 2006; SIPSER, 2012):

- Q é um conjunto finito de estados;
- Σ é um alfabeto finito de símbolos de entrada;
- $\delta : Q \times \Sigma \rightarrow Q$ é a **função de transição**, que associa a cada par formado por um estado e um símbolo o estado para o qual o autômato transita;
- $q_0 \in Q$ é o estado inicial;
- $F \subseteq Q$ é o conjunto de estados finais (ou de aceitação).

A função δ descreve o comportamento do autômato ao ler um único símbolo. Para representar o processamento de uma cadeia inteira, define-se sua extensão recursiva $\hat{\delta} : Q \times \Sigma^* \rightarrow Q$, dada por (HOPCROFT *et al.*, 2006):

$$\begin{aligned}\hat{\delta}(q, \varepsilon) &= q, \\ \hat{\delta}(q, wa) &= \delta(\hat{\delta}(q, w), a),\end{aligned}$$

em que $w \in \Sigma^*$ e $a \in \Sigma$. A função estendida indica, portanto, o estado em que o autômato se encontra após processar uma cadeia w a partir do estado q . A **linguagem reconhecida** (ou aceita) por um AFD M , denotada $L(M)$, é o conjunto de todas as

cadeias que conduzem o autômato, partindo do estado inicial, a algum estado de aceitação:

$$L(M) = \{ w \in \Sigma^* \mid \hat{\delta}(q_0, w) \in F \}.$$

As linguagens reconhecidas por algum AFD são denominadas **linguagens regulares** (SIPSER, 2012).

A representação gráfica de um AFD é feita por meio de um **diagrama de estados**, no qual os estados são representados por círculos, as transições por arestas rotuladas com os símbolos correspondentes, o estado inicial é apontado por uma seta sem origem e os estados de aceitação são marcados com círculos duplos. A título de exemplo, considere o AFD apresentado na Figura 2, que reconhece a linguagem das cadeias sobre $\{0, 1\}$ que contêm 01 como subcadeia (HOPCROFT *et al.*, 2006).

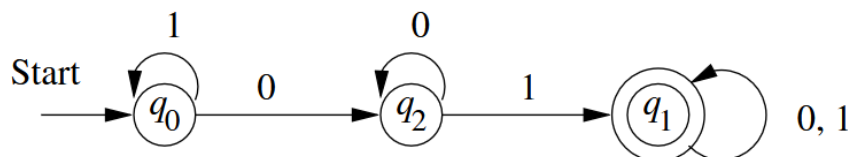


FIGURA 2 – AFD que reconhece cadeias contendo 01 como subcadeia

FONTE: Hopcroft *et al.* (2006)

Formalmente, o autômato da Figura 2 é descrito como $M = (Q, \Sigma, \delta, q_0, F)$, em que $Q = \{q_0, q_1, q_2\}$, $\Sigma = \{0, 1\}$, $F = \{q_1\}$ e a função δ é dada pelas transições $\delta(q_0, 0) = q_2$, $\delta(q_0, 1) = q_0$, $\delta(q_2, 0) = q_2$, $\delta(q_2, 1) = q_1$, $\delta(q_1, 0) = q_1$ e $\delta(q_1, 1) = q_1$. Cada estado codifica uma informação distinta sobre o histórico da leitura: q_0 indica que ainda não foi lido nenhum 0 (ou que o último símbolo lido foi 1), q_2 indica que o último símbolo lido foi 0 e q_1 indica que a subcadeia 01 já foi identificada (HOPCROFT *et al.*, 2006).

A título de ilustração, ao processar a cadeia 1101, o autômato realiza a sequência de transições $q_0 \xrightarrow{1} q_0 \xrightarrow{1} q_0 \xrightarrow{0} q_2 \xrightarrow{1} q_1$, atingindo o estado de aceitação q_1 , de modo que a cadeia é aceita. Por outro lado, ao processar a cadeia 111, o autômato permanece em q_0 ao longo de toda a leitura e a cadeia é rejeitada por não terminar em estado final. Verifica-se que $L(M)$ corresponde exatamente ao conjunto das cadeias sobre $\{0, 1\}$ que contêm a subcadeia 01, em conformidade com a especificação informal apresentada.

2.4 AUTÔMATO FINITO NÃO-DETERMINÍSTICO

O autômato finito determinístico apresentado na seção anterior caracteriza-se por possuir, para cada combinação de estado e símbolo de entrada, uma única transição definida. Em diversas situações, no entanto, é mais natural descrever um reconhecedor permitindo que, em certos estados, mais de uma transição esteja disponível para

o mesmo símbolo, ou ainda que, em outros, não haja qualquer transição definida (HOPCROFT *et al.*, 2006; SIPSER, 2012). Tal flexibilidade dá origem ao **autômato finito não-determinístico** (AFN), modelo que, embora distinto em comportamento, reconhece a mesma classe de linguagens que os AFDs.

A diferença essencial em relação ao AFD reside na natureza da função de transição. Enquanto, em um AFD, a aplicação de δ a um par estado-símbolo retorna um único estado, em um AFN ela retorna um **conjunto** de estados, possivelmente vazio. Isso significa que, ao processar um símbolo a partir de um dado estado, o AFN pode prosseguir por qualquer um dos estados indicados (caso haja mais de um) ou travar (caso o conjunto seja vazio). Intuitivamente, imagina-se que o autômato investiga simultaneamente todas as escolhas possíveis: o caminho computacional que, ao final da leitura, atinge um estado de aceitação determina a aceitação da cadeia, mesmo que outros caminhos tenham travado ou terminado em estados não-finais (SIPSER, 2012).

Formalmente, um **autômato finito não-determinístico** é uma quintupla $N = (Q, \Sigma, \delta, q_0, F)$, na qual (HOPCROFT *et al.*, 2006; SIPSER, 2012):

- Q é um conjunto finito de estados;
- Σ é um alfabeto finito de símbolos de entrada;
- $\delta : Q \times \Sigma \rightarrow \mathcal{P}(Q)$ é a **função de transição**, que associa a cada par estado-símbolo um subconjunto de Q , sendo $\mathcal{P}(Q)$ o conjunto das partes de Q ;
- $q_0 \in Q$ é o estado inicial;
- $F \subseteq Q$ é o conjunto de estados finais.

A extensão da função de transição a cadeias, denotada $\hat{\delta}$, é definida recursivamente de modo análogo ao caso determinístico, levando em consideração que cada passo pode produzir múltiplos estados (HOPCROFT *et al.*, 2006):

$$\begin{aligned}\hat{\delta}(q, \varepsilon) &= \{q\}, \\ \hat{\delta}(q, wa) &= \bigcup_{p \in \hat{\delta}(q, w)} \delta(p, a),\end{aligned}$$

em que $w \in \Sigma^*$ e $a \in \Sigma$. A função estendida indica, portanto, o conjunto de todos os estados em que o autômato pode encontrar-se após processar a cadeia w a partir do estado q . A **linguagem aceita** por um AFN N , denotada $L(N)$, é o conjunto das cadeias para as quais existe ao menos um caminho computacional terminando em estado de aceitação:

$$L(N) = \{ w \in \Sigma^* \mid \hat{\delta}(q_0, w) \cap F \neq \emptyset \}.$$

Em outras palavras, basta que uma das possíveis sequências de transições conduza a um estado final para que a cadeia seja aceita (HOPCROFT *et al.*, 2006; SIPSER, 2012).

A título de exemplo, considere o AFN apresentado na Figura 3, que reconhece a linguagem das cadeias sobre $\{0, 1\}$ que terminam em 01 (HOPCROFT *et al.*, 2006).

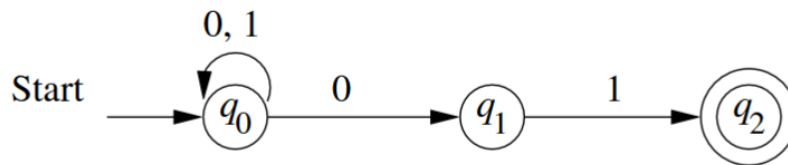


FIGURA 3 – AFN que reconhece cadeias terminadas em 01

FONTE: Hopcroft *et al.* (2006)

Esse autômato é descrito formalmente como $N = (\{q_0, q_1, q_2\}, \{0, 1\}, \delta, q_0, \{q_2\})$, em que a função de transição é dada por $\delta(q_0, 0) = \{q_0, q_1\}$, $\delta(q_0, 1) = \{q_0\}$, $\delta(q_1, 1) = \{q_2\}$ e os demais valores correspondem ao conjunto vazio. As duas características que evidenciam o não-determinismo ficam aparentes nessa especificação: o estado q_0 admite duas transições distintas para o símbolo 0 (o autômato pode permanecer em q_0 ou avançar para q_1), enquanto os estados q_1 e q_2 apresentam transições ausentes para certos símbolos.

Para ilustrar o funcionamento, considere o processamento da cadeia 001. Partindo de $\hat{\delta}(q_0, \varepsilon) = \{q_0\}$, calcula-se sucessivamente $\hat{\delta}(q_0, 0) = \{q_0, q_1\}$, $\hat{\delta}(q_0, 00) = \{q_0, q_1\}$ e $\hat{\delta}(q_0, 001) = \{q_0, q_2\}$. Como $q_2 \in F$, a cadeia é aceita. Por outro lado, ao processar a cadeia 010, obtém-se ao final $\hat{\delta}(q_0, 010) = \{q_0, q_1\}$, conjunto que não contém o estado final q_2 , de modo que a cadeia é rejeitada.

Embora o não-determinismo confira maior flexibilidade aparente na descrição de linguagens, demonstra-se que toda linguagem reconhecida por um AFN também pode ser reconhecida por algum AFD (HOPCROFT *et al.*, 2006; SIPSER, 2012). Esse resultado, fundamentado na chamada **construção de subconjuntos**, é apresentado na seção seguinte e justifica o fato de ambos os modelos definirem exatamente a mesma classe de linguagens: as linguagens regulares.

2.5 EQUIVALÊNCIA ENTRE AFD E AFN

Embora o autômato finito não-determinístico ofereça maior flexibilidade na descrição de linguagens, demonstra-se que essa flexibilidade não amplia a classe de linguagens reconhecíveis. Mais precisamente, vale o seguinte resultado central da teoria dos autômatos: uma linguagem L é reconhecida por algum AFD se, e somente se, é reconhecida por algum AFN (HOPCROFT *et al.*, 2006; SIPSER, 2012). Em outras

palavras, AFDs e AFNs são modelos **equivalentes em poder de reconhecimento**, ainda que possam diferir significativamente em número de estados e em facilidade de projeto.

A demonstração desse resultado é constituída por duas direções. A primeira, e mais imediata, consiste em observar que todo AFD pode ser interpretado como um AFN: basta tomar a função de transição do AFD e, para cada par estado-símbolo, considerar o conjunto unitário formado pelo único destino existente. Como o AFN resultante simula passo a passo o AFD original, ambos reconhecem a mesma linguagem (HOPCROFT *et al.*, 2006). A segunda direção, conhecida como **construção de subconjuntos** (*subset construction*), é menos trivial e fornece um procedimento algorítmico para converter qualquer AFN em um AFD equivalente.

A construção de subconjuntos parte da seguinte ideia: em um AFN, após ler uma cadeia w , o autômato pode encontrar-se em qualquer um dos estados pertencentes a $\hat{\delta}(q_0, w)$. O AFD equivalente, por sua vez, simula esse comportamento mantendo, em cada um de seus estados, o conjunto de estados do AFN em que a computação poderia estar a partir da leitura realizada até aquele momento. Os estados do AFD são, portanto, subconjuntos do conjunto de estados do AFN (HOPCROFT *et al.*, 2006).

Formalmente, dado um AFN $N = (Q_N, \Sigma, \delta_N, q_0, F_N)$, constrói-se o AFD equivalente $D = (Q_D, \Sigma, \delta_D, q_{0,D}, F_D)$ por meio das seguintes definições (HOPCROFT *et al.*, 2006):

- $Q_D = \mathcal{P}(Q_N)$, ou seja, cada estado do AFD é um subconjunto de estados do AFN;
- $q_{0,D} = \{q_0\}$, isto é, o estado inicial do AFD corresponde ao conjunto que contém apenas o estado inicial do AFN;
- $F_D = \{S \in Q_D \mid S \cap F_N \neq \emptyset\}$, ou seja, um subconjunto é considerado estado de aceitação do AFD se contiver ao menos um estado de aceitação do AFN;
- $\delta_D(S, a) = \bigcup_{p \in S} \delta_N(p, a)$ para cada $S \in Q_D$ e cada $a \in \Sigma$, isto é, a transição do AFD a partir de um subconjunto S é a união das transições que o AFN executaria a partir de cada estado em S .

Embora $\mathcal{P}(Q_N)$ contenha $2^{|Q_N|}$ elementos no caso geral, em geral apenas uma pequena fração desses subconjuntos é efetivamente alcançável a partir do estado inicial. Na prática, o procedimento é aplicado de forma incremental: parte-se de $\{q_0\}$ e, a cada passo, computam-se as transições para os subconjuntos ainda não explorados, até que nenhum novo subconjunto seja gerado. A correção da construção é assegurada pelo seguinte teorema, demonstrado em Hopcroft *et al.* (2006): *uma linguagem $L \subseteq \Sigma^*$ é reconhecida por algum AFN se, e somente se, é reconhecida por algum AFD.*

A título de ilustração, considere novamente o AFN da Figura 3, que reconhece as cadeias sobre $\{0, 1\}$ terminadas em 01. Aplicando a construção de subconjuntos a esse autômato, parte-se do estado inicial $\{q_0\}$ e calculam-se as transições conforme o procedimento descrito. Tem-se, por exemplo, $\delta_D(\{q_0\}, 0) = \delta_N(q_0, 0) = \{q_0, q_1\}$ e $\delta_D(\{q_0\}, 1) = \{q_0\}$. Continuando o processo, geram-se sucessivamente os demais subconjuntos alcançáveis até que o AFD se estabilize. O resultado final é apresentado na Figura 4.

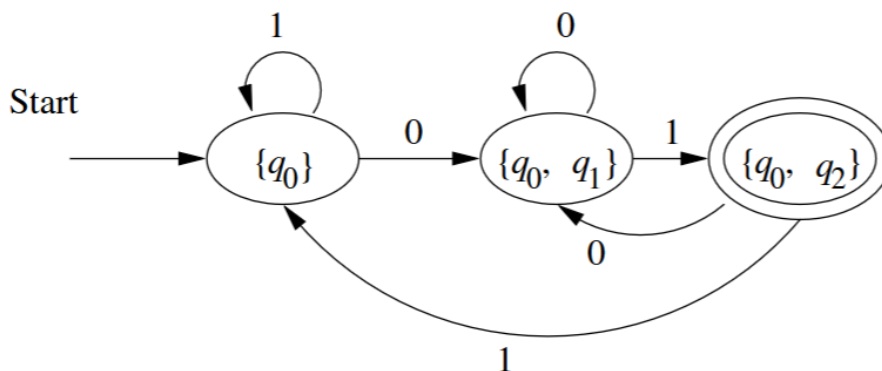


FIGURA 4 – AFD construído a partir do AFN da Figura 3 pela construção de subconjuntos

FONTE: Hopcroft *et al.* (2006)

Observa-se que o AFD resultante possui apenas três estados ($\{q_0\}$, $\{q_0, q_1\}$ e $\{q_0, q_2\}$), ainda que o conjunto das partes de Q_N contivesse oito subconjuntos possíveis. O estado $\{q_0, q_2\}$ é o único estado de aceitação, pois é o único subconjunto acessível que contém q_2 , estado final do AFN de origem. Verifica-se diretamente que $L(D) = L(N)$, isto é, o AFD construído reconhece exatamente a mesma linguagem do AFN de partida (HOPCROFT *et al.*, 2006).

Vale observar, por fim, que, no pior caso, a construção de subconjuntos pode produzir um AFD com 2^n estados a partir de um AFN com n estados, ainda que, na prática, o número de estados acessíveis costume ser comparável ao do AFN original (HOPCROFT *et al.*, 2006). Essa diferença, contudo, não altera a equivalência em poder expressivo: ambos os modelos definem exatamente a classe das linguagens regulares.

2.6 REPRESENTAÇÕES DE AUTÔMATOS

A definição formal de autômato finito como uma quintupla, apresentada nas seções anteriores, é suficiente para caracterizar matematicamente o objeto, mas não é necessariamente a forma mais prática de descrevê-lo no dia a dia. Na literatura e em ferramentas educacionais, são utilizadas três representações distintas e complementares, cada uma adequada a um propósito específico (HOPCROFT *et al.*, 2006; SIPSER,

2012). Esta seção apresenta essas representações, evidenciando as vantagens e limitações de cada uma.

A primeira forma é a **notação formal**, em que o autômato é descrito explicitamente como a quintupla $M = (Q, \Sigma, \delta, q_0, F)$, listando-se cada um de seus componentes. Trata-se da representação mais precisa e desambígua, sendo indispensável em demonstrações matemáticas e em provas de equivalência. Por outro lado, sua leitura direta é trabalhosa: a função de transição δ , em particular, costuma ser apresentada como uma enumeração de pares $\delta(q, a) = q'$, que rapidamente se torna extensa à medida que cresce o número de estados (HOPCROFT *et al.*, 2006).

A segunda forma é o **diagrama de estados**, já utilizado nas Figuras 2 e 3 das seções anteriores. Nessa representação, cada estado corresponde a um nó (círculo simples para estados comuns ou círculo duplo para estados de aceitação), as transições são arestas rotuladas com os símbolos do alfabeto, e o estado inicial é apontado por uma seta sem origem (HOPCROFT *et al.*, 2006; SIPSER, 2012). O diagrama de estados é o formato preferido em contextos didáticos e em ferramentas como o JFLAP (RODGER; FINLEY, 2006), pois permite que o leitor reconheça visualmente padrões como ciclos, estados de absorção e simetrias. Sua principal limitação é a dificuldade de manipulação textual: o diagrama é uma imagem e, portanto, não pode ser facilmente versionado, comparado ou processado por ferramentas automatizadas.

A terceira forma é a **tabela de transição**, que organiza a função δ em uma matriz bidimensional cujas linhas correspondem aos estados e cujas colunas correspondem aos símbolos do alfabeto (HOPCROFT *et al.*, 2006). A Tabela 2 ilustra essa representação para o AFD da Figura 2, que reconhece cadeias contendo 01 como subcadeia. A seta ($\rightarrow$) marca o estado inicial e o asterisco (*) destaca o estado de aceitação, conforme convenção adotada por Hopcroft *et al.* (2006).

TABELA 2 – Tabela de transição do AFD que reconhece cadeias contendo 01

	0	1
$\rightarrow q_0$	q_2	q_0
q_2	q_2	q_1
$*q_1$	q_1	q_1

FONTE: Adaptado de Hopcroft *et al.* (2006)

A leitura da tabela é direta: cada célula (q, a) contém o estado $\delta(q, a)$ para o qual o autômato transita ao ler o símbolo a a partir do estado q . Essa representação combina precisão formal e leitura sistemática, sendo compacta, permitindo verificar facilmente se a função de transição está completa (no caso do AFD, todas as células devem estar preenchidas) e sendo particularmente adequada para o processamento

programático, pois pode ser diretamente codificada como uma matriz ou estrutura de dados equivalente (HOPCROFT *et al.*, 2006; SIPSER, 2012). Para AFNs, a tabela de transição segue o mesmo formato, com a diferença de que cada célula contém um *conjunto* de estados em vez de um único estado, podendo, inclusive, conter o conjunto vazio quando não houver transição definida (HOPCROFT *et al.*, 2006).

As três representações apresentadas (notação formal, diagrama de estados e tabela de transição) são complementares e equivalentes entre si, descrevendo o mesmo objeto matemático sob perspectivas distintas. A escolha entre elas depende do contexto de uso: o rigor formal favorece a notação como quintupla; a comunicação didática favorece o diagrama; a manipulação computacional favorece a tabela. A ferramenta proposta neste trabalho explora essa equivalência ao oferecer uma quarta forma de representação, baseada em uma linguagem de domínio específico textual, que combina a precisão da notação formal com a estruturação tabular, permitindo simultaneamente a renderização do diagrama de estados e o processamento programático da descrição.

2.7 SIMULAÇÃO DE AUTÔMATOS

A definição formal de um autômato finito, vista nas seções anteriores, descreve sob a forma de quintupla os elementos que compõem a máquina, mas não esclarece, por si só, como verificar se uma cadeia específica é aceita ou rejeitada. Essa verificação, conhecida como **simulação**, consiste em executar passo a passo a função de transição δ a partir do estado inicial, consumindo cada símbolo da cadeia de entrada e observando se a configuração final corresponde a um estado de aceitação (HOPCROFT *et al.*, 2006; SIPSER, 2012). Em termos pedagógicos, a simulação cumpre um papel análogo ao da execução de um programa em uma linguagem de programação, pois permite que o estudante observe o comportamento do autômato projetado e confronte-o com o comportamento esperado (MORAZÁN; ANTUNEZ, 2014).

Morazán e Antunez (2014) argumentam que disciplinas introdutórias de linguagens formais, tradicionalmente, exigem que estudantes projetem máquinas e provem sua correção sem oferecer mecanismos para experimentar com os artefatos produzidos. Os autores defendem que essa ausência de retroalimentação imediata contraria a forma como o estudante de Ciência da Computação aprende a programar, em que compiladores e interpretadores fornecem mensagens de erro e advertências durante o desenvolvimento. A consequência observada é que estudantes inexperientes submetem com frequência soluções incorretas, mesmo para exercícios relativamente simples, e dedicam esforço a tentativas de prova de correção sobre projetos defeituosos (MORAZÁN; ANTUNEZ, 2014). Para mitigar esse problema, os autores propõem que a teoria da computação seja ensinada com ferramentas capazes de *construir* computações, isto é, ambientes em que a descrição formal possa ser traduzida em uma representação

executável e submetida a testes.

Formalmente, simular um AFD $M = (Q, \Sigma, \delta, q_0, F)$ sobre uma cadeia $w = a_1 a_2 \dots a_n \in \Sigma^*$ corresponde a calcular a sequência de configurações $q_0, q_1, \dots, q_n$, em que $q_i = \delta(q_{i-1}, a_i)$ para $1 \leq i \leq n$. A cadeia w é aceita se, e somente se, $q_n \in F$ (HOPCROFT *et al.*, 2006). Para um AFD, a simulação é determinística e linear: cada símbolo lido conduz a exatamente um próximo estado, de modo que o custo computacional cresce proporcionalmente ao comprimento da cadeia. Essa propriedade torna o AFD particularmente adequado para implementações eficientes, pois pode ser realizado por uma estrutura de dados simples, como uma tabela de transição indexada pelo par (estado, símbolo) (SIPSER, 2012).

A simulação de um AFN, em contraste, exige tratamento adicional, pois a função de transição δ pode levar um mesmo par (estado, símbolo) a um conjunto de estados, e transições ε permitem mudança de estado sem consumir símbolos da entrada. Conforme descrito por Morazán e Antunez (2014) ao apresentar a função *apply-sm* da biblioteca FSM, a simulação de uma máquina não determinística é realizada por meio de uma busca em largura sobre todos os caminhos possíveis que o autômato pode seguir ao consumir a cadeia. Em cada passo, o simulador mantém o conjunto de estados alcançáveis e, ao final da entrada, verifica se algum desses estados pertence ao conjunto de aceitação. Alternativamente, conforme Hopcroft *et al.* (2006), o AFN pode ser convertido previamente em um AFD equivalente pelo algoritmo de construção de subconjuntos, transferindo a simulação para o caso determinístico ao custo de uma possível expansão exponencial do número de estados.

Além de produzir um veredito de aceitação ou rejeição, uma ferramenta de simulação útil para o ensino deve expor o *caminho percorrido* pelo autômato, isto é, a sequência de configurações intermediárias atravessadas durante o processamento da cadeia. Morazán e Antunez (2014) denominam essa funcionalidade *show-transitions* e destacam sua importância para que o estudante identifique exatamente em que ponto sua máquina diverge do comportamento esperado. Essa visualização do *trace* de execução substitui o exercício manual de acompanhar transições em papel, propenso a erros, por uma observação direta do comportamento da máquina, em consonância com a prática de depuração comum no desenvolvimento de *software*.

A relevância da simulação ganha ainda mais destaque quando se considera o fenômeno descrito por Morazán e Antunez (2014) de que estudantes, sem ambiente de teste, frequentemente partem para demonstrações formais sobre projetos incorretos, com perda de tempo e desmotivação como consequência. Esse argumento é central para o presente trabalho: a ferramenta proposta busca oferecer simulação visual interativa diretamente acoplada à descrição textual do autômato, de forma que o estudante possa, a partir de uma única especificação, examinar a estrutura do diagrama de estados, executar cadeias de teste e acompanhar a evolução das configurações,

recebendo retroalimentação imediata sobre o comportamento da máquina.

Cabe ainda diferenciar dois sentidos do termo simulação que aparecem na literatura. Em sentido estrito, simular um autômato é executar sua função de transição sobre uma entrada específica, como descrito acima. Em sentido mais amplo, ferramentas como o JFLAP (RODGER; FINLEY, 2006) e a biblioteca FSM (MORAZÁN; ANTUNEZ, 2014) oferecem também a possibilidade de *testar* um autômato com múltiplas cadeias geradas aleatoriamente, comparar duas máquinas quanto à equivalência de suas linguagens e visualizar transformações construtivas, como a conversão de AFN em AFD. Essas operações estendem a noção básica de simulação para um conjunto mais rico de recursos didáticos, ao qual se pode atribuir o termo *ambiente de experimentação*. A ferramenta proposta neste trabalho posiciona-se nesse espectro, oferecendo, em primeiro plano, a simulação visual passo a passo de autômatos finitos descritos por meio de uma linguagem de domínio específico, conforme será detalhado no Capítulo 3.

2.8 APLICAÇÕES DE AUTÔMATOS

Apesar de serem objetos teóricos, autômatos finitos não são uma abstração restrita ao ambiente acadêmico. Hopcroft *et al.* (2006) destacam que esse modelo computacional sustenta o funcionamento de diversas categorias de *software* e *hardware* utilizadas no dia a dia, sempre que um sistema precisa registrar, em um número finito de configurações, a parte relevante de seu histórico para decidir qual ação tomar diante da próxima entrada. Esta seção apresenta as principais áreas de aplicação descritas pelos autores e estabelece a ponte entre os conceitos formais discutidos até aqui e os capítulos seguintes deste trabalho.

Hopcroft *et al.* (2006) agrupam as aplicações dos autômatos finitos em quatro grandes categorias:

- **Projeto e verificação de circuitos digitais.** Sistemas digitais com um número finito de estados internos podem ser modelados diretamente como autômatos, permitindo que o comportamento do circuito seja descrito de forma rigorosa e verificado antes da implementação física. Cada estado do autômato corresponde a uma configuração possível do circuito, e cada transição corresponde à resposta do *hardware* a um sinal de entrada (HOPCROFT *et al.*, 2006).
- **Análise léxica de compiladores.** O componente responsável por dividir o texto-fonte em unidades léxicas (identificadores, palavras-chave, operadores e literais) é tipicamente implementado como um autômato finito. Hopcroft *et al.* (2006) ilustram esse uso com um autômato cuja função é reconhecer a palavra-chave `then`: cada estado corresponde a um prefixo da palavra (vazio, `t`, `th`, `the`, `then`), e a sequência de transições percorrida pela cadeia de entrada determina se o lexema

foi encontrado. Esse mesmo princípio se generaliza para o reconhecimento de qualquer conjunto finito de *tokens* descrito por expressões regulares.

- **Busca de padrões em grandes volumes de texto.** Sistemas que percorrem coleções extensas de documentos, páginas *web* ou arquivos em busca de palavras, expressões ou padrões mais complexos costumam compilar essas consultas em autômatos finitos, executando a varredura em tempo proporcional ao tamanho da entrada (HOPCROFT *et al.*, 2006). Utilitários clássicos do ambiente Unix, como *grep*, e bibliotecas modernas de expressões regulares apoiam-se diretamente nessa equivalência entre expressões regulares e autômatos finitos.
- **Verificação de sistemas com número finito de estados.** Protocolos de comunicação, protocolos criptográficos e outros sistemas reativos podem ser modelados como autômatos para que propriedades como ausência de *deadlock*, alcançabilidade de estados ou conformidade com uma especificação sejam verificadas automaticamente (HOPCROFT *et al.*, 2006).

A presença pervasiva dos autômatos finitos nessas áreas decorre de uma característica central do modelo: o número fixo de estados permite que sua representação seja implementada com uma quantidade limitada de recursos, seja como um circuito de *hardware*, seja como uma estrutura de dados em memória (HOPCROFT *et al.*, 2006). Esse equilíbrio entre simplicidade e poder expressivo explica por que o modelo permanece relevante mesmo após décadas de evolução das ferramentas de *software*.

Para o presente trabalho, a aplicação de maior interesse é a análise léxica, pois constitui o elo direto entre a teoria dos autômatos finitos e a construção de compiladores e interpretadores para linguagens de programação. A linguagem de domínio específico proposta nesta monografia, descrita no Capítulo 3, será analisada lexicamente por um autômato finito gerado a partir das regras léxicas da própria linguagem, em uma aplicação concreta dos conceitos apresentados neste capítulo.

3 COMPILADORES E LINGUAGENS DE DOMÍNIO ESPECÍFICO

texto breve aqui depois

3.1 CONCEITOS DE COMPILADORES

3.2 ANÁLISE LÉXICA

3.3 ANÁLISE SINTÁTICA

3.4 ANÁLISE SEMÂNTICA

3.5 AST (ÁRVORE SINTÁTICA ABSTRATA)

3.6 LINGUAGENS DE DOMÍNIO ESPECÍFICO (DSL)

3.7 CONSTRUÇÃO DE COMPILADORES (FERRAMENTAS)

3.8 DIAGNÓSTICO DE ERROS

3.9 DSL NO ENSINO

4 RUST

REFERÊNCIAS

- DIVERIO, T. A.; MENEZES, P. B. **Teoria da Computação: Máquinas Universais e Computabilidade**. 3. ed. Porto Alegre: Bookman, 2011.
- HOPCROFT, J. E.; MOTWANI, R.; ULLMAN, J. D. **Introduction to Automata Theory, Languages, and Computation**. 3. ed. Boston, MA: Pearson Addison-Wesley, 2006.
- LEWIS, H. R.; PAPADIMITRIOU, C. H. **Elements of the Theory of Computation**. 2. ed. Upper Saddle River, NJ: Prentice Hall, 1998.
- MORAZÁN, M. T.; ANTUNEZ, R. Functional automata: Formal languages for computer science students. In: **Proceedings of the 3rd International Workshop on Trends in Functional Programming in Education (TFPIE 2014)**. [S.l.]: Open Publishing Association, 2014. (Electronic Proceedings in Theoretical Computer Science, v. 170), p. 19–32.
- RODGER, S. H.; FINLEY, T. W. **JFLAP: An Interactive Formal Languages and Automata Package**. Sudbury, MA: Jones and Bartlett Publishers, 2006.
- SIPSER, M. **Introduction to the Theory of Computation**. 3. ed. Boston, MA: Cengage Learning, 2012.
- VIEIRA, N. J. **Introdução aos Fundamentos da Computação: Linguagens e Máquinas**. São Paulo: Thomson Learning, 2006.